GENEROUS MOTORS

DRIVE THE CAR. FUND THE CAUSE.

Demo Guide & Build Proposal

A working, clickable demonstration of the Generous Motors platform — public site, member accounts, tiered promotions, and a full admin operations dashboard — plus the production architecture, timeline, and investment to ship it for real.

LIVE DEMO

https://gmdemo-orcin.vercel.app

Prepared July 2026 · Demo data only — nothing on the demo represents real entries, orders, or donations.

Everything that's clickable today

The demo persists all admin changes and sign-ins to the browser's local storage — every desk works, nothing needs a server. Open it on desktop for the admin, and on a phone for the buyer experience.

Public site

/	Homepage — full-bleed hero with promo countdown, winners wall with Latest Winner card, pricing, live-draw block, charity band, FAQ.
/tickets	The buy page — gallery with swipe, bundle grid with Buy Now per bundle, membership tab, promo pickup, spec sheet, FAQ + Official Rules.
/tickets?promo=VIP3X	Same page arriving from an SMS link — 3X promo banner, strikethrough ticket counts, 'Promo closes in' countdown.
/tickets?utm_campaign=ads	Same page arriving from a paid ad — the advertising tier multiplier picks up from the UTM parameter.
/account/login	Passwordless sign-in — enter any email; the one-time code appears in the on-page demo inbox.
/account	Member dashboard — entries with real ticket numbers, current giveaway, member 4X offer.
/account/tickets	Member buy page — every bundle shows 4-for-1 strikethrough counts, applied automatically.
/winners /partners	Winner archive and the partner logo wall (charity partners + brand sponsors).
/blog /about /live	Field notes (admin-publishable), the trust story, and the live-draw page.
/rules /contact	Official Rules per Fla. Stat. § 849.0935, and the contact form with auto-confirmation.

Admin dashboard — /admin

/admin/promotions	The tier framework: per-channel multipliers, promo codes, UTM triggers, end dates with independent countdowns, marketing copy.
/admin/attribution	Who came from where and who bought — visits, purchases, conversion and revenue per channel, with the raw triggers.
/admin/content	Copy CMS — 41 fields organized by page (hero words, section headings, popup copy, footer legal), live on publish.
/admin/blog	Article editor — Markdown or raw HTML with preview, SEO title/description/OG image, SERP preview, publish to /blog.
/admin/cycles	Cycle desk — draw date (feeds every countdown), entry cap, charity partner selection, and the partner registry.
/admin/tickets	Barrel printing — filter any cycle + purchase date range, generate the A3 black & white ticket sheets PDF.
/admin/sms	SMS blasts — compose, attach a promo (adds the trigger link), send to the list, delivery stats per blast.
/admin/newsletter	The weekly newsletter — compose, attach a promo, send, open/click stats per issue.

FIVE-MINUTE WALKTHROUGH

1. Open /tickets?promo=VIP3X and buy a bundle — note the banner, strikethrough counts, and promo countdown.
2. Open /admin/attribution — the visit and purchase are logged under SMS subscribers with the trigger.
3. In /admin/promotions, change the SMS multiplier to 5X and save — reload the promo link and watch it update.
4. In /admin/cycles, move the draw date — the header, hero, and buy-box countdowns all follow.
5. In /admin/tickets, generate the Cycle 12 A3 sheet PDF — logo, holder name, phone, ticket number, 15 per sheet.

PART 02 · PRODUCTION BUILD

Architecture — simple, one place to manage

Shopify owns money and merchandising. A custom Next.js storefront on Vercel owns the brand experience. Vercel Postgres owns what Shopify can't: ticket numbers, promotions, members, and attribution. The admin dashboard from the demo becomes the single pane of glass; orders are worked in Shopify admin.

The stack

- **Shopify —** checkout, orders, refunds, subscription billing, product catalog, and the headless CMS (metaobjects) for content, blog posts, winners, and cycles.
- **Next.js —** custom storefront on Vercel; the exact pages in this demo, reading Shopify via the Storefront API. Edge runtime for reads (promo resolution, content), Node serverless for webhooks and PDF generation.
- **Vercel Postgres —** tickets, promotions, users/OTP sessions, attribution events. The system of record for everything drawn from the drum.
- **Postscript + Klaviyo/SendGrid —** SMS list + blasts (Postscript), email newsletter (Klaviyo or SendGrid). Campaigns are composed in our admin and sent through their APIs, so day-to-day stays in one place.

Catalog modeling on Shopify

- **Memberships —** one product per tier (Essential / Premium / VIP), each with a selling plan. Shopify Subscriptions works headless through the Storefront Cart API: the cart line carries a sellingPlanId, and checkout handles recurring billing. Requires the unauthenticated_read_selling_plans scope.
- **Ticket bundles —** the raffle-platform standard is a unique SKU per bundle so the webhook can map SKU to entry count deterministically. Recommendation: one 'Cycle N Tickets' product with one variant per bundle (1/5/10/25/50/100) — single page, clean reporting, promo-friendly — rather than six separate products. Separate products only if a bundle ever needs its own imagery or ad landing page.
- **Promotions —** entry multipliers live in Postgres (they change entries, not price) exactly as in the demo; price discounts, when used, are Shopify automatic discounts so checkout stays truthful.

Ticket generation — the part that cannot fail

Tickets are generated by the orders/paid webhook. The design goal is exactly-once issuance: no duplicate numbers, no missing tickets, under retries, concurrency, and partial failures.

- HMAC-verify every webhook; respond fast; process in a Node serverless function.
- **Idempotency:** orders table with a UNIQUE constraint on shopify_order_id. The insert is ON CONFLICT DO NOTHING — a retried or duplicated webhook becomes a no-op. Shopify retries for 48h, and that's safe here.
- **Allocation:** ticket numbers come from a per-cycle counter row updated atomically (UPDATE … SET last = last + qty RETURNING last) inside the same transaction as the ticket rows. The row lock serializes concurrent orders — no collisions; the transaction guarantees no gaps if anything fails mid-way.
- **Backstop:** UNIQUE(cycle_id, ticket_number) as the schema-level backstop. If the impossible happens, the database refuses the duplicate rather than issuing it.
- **Reconciliation:** a reconciliation cron sweeps paid Shopify orders against the tickets table every 15 minutes;

anything missed (dropped webhook, outage) is generated then, and drift alerts the team. This closes the 'webhook never arrived' hole.

- **Notification:** the confirmation email with ticket numbers fires only after the transaction commits.

Analytics, attribution & the management split

GA4, Meta Pixel & UTM tracking

- GA4 and Meta Pixel load in the Next.js root layout; standard ecommerce events (view_item, add_to_cart, begin_checkout) fire client-side on the storefront.
- checkout runs on Shopify's domain, so a Shopify custom Web Pixel (Settings, then Customer events) fires checkout and purchase events to the same GA4 / Meta IDs — one property, no double counting.
- GA4 cross-domain linking connects storefront and checkout sessions; UTM and promo parameters are captured on landing (exactly as the demo does), written to cart attributes, and flow onto the order.
- the webhook writes the source-to-order mapping to Postgres, so the Attribution desk in the admin shows real conversion and revenue per channel — same screen you see in the demo.
- Meta Conversions API fires server-side from the webhook for ad-blocker-proof purchase signals.

Who manages what

ADMIN DASHBOARD (OURS — EVERYTHING SITE-RELATED)

Promotions & multipliers · attribution · site copy · blog · winners · cycles & partners · ticket sheet printing · SMS blasts · newsletter issues.
Content is stored in Shopify metaobjects, so it is also editable from Shopify admin — one source of truth, two doors.

SHOPIFY ADMIN (MONEY)

Orders, refunds, chargebacks, customer service, product/price changes, subscription billing states, payouts.

POSTSCRIPT / KLAVIYO (DELIVERY RAILS)

Compliance, deliverability, carrier relationships. Campaigns are composed in our admin and dispatched via their APIs.

Timeline & investment

Four weeks, plus a five-day buffer

- **Week 1 —** Shopify store setup — products, variants, selling plans, webhooks. Next.js storefront scaffold with the demo's pages. Postgres schema. Email OTP auth.
- **Week 2 —** purchase flows through the Storefront Cart API (bundles + subscriptions), the ticket-generation webhook engine with reconciliation, promotions engine, checkout + post-purchase upsells.
- **Week 3 —** full admin suite on live data, campaign sending through Postscript and Klaviyo/SendGrid, analytics + attribution wiring. Public beta delivered.
- **Week 4 —** hardening, load and security pass, content load, DNS cutover, production deployment.
- **Buffer (5 days) —** review rounds, fixes, and the unexpected.

Investment — $7,875 fixed

Excludes Vercel, Shopify, SendGrid/Klaviyo, Postscript subscriptions and any other third-party costs.

- $1,575 (20%) — advance by bank transfer, schedules the build.
- $3,937.50 (50%) — at Week 3, on delivery of the public beta.

- $2,362.50 (30%) — after successful production deployment plus 48 hours with no issue or incident.

SOURCES — PRODUCTION RESEARCH

Shopify: Manage subscription products on storefronts (shopify.dev, Storefront Cart API + sellingPlanId).
ViralSweep: Shopify raffle ticket setup — unique SKU per bundle/variant.
Shopify Help: custom Web Pixels for GA4/Meta on checkout; GA4 cross-domain measurement guides.